Computer Science & IT



Data Structure & Programming

Tree

Lecture No. 04



By- Abhishek Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic
- Topic
- Topic

Counting < unlabeled, Labeled

7 - 8 marks

Sequential representation

General tree Representation

Slide

Topics to be Covered



- Topic — Binary tree Node
- Topic — Traversal — Inorder, preorder, postorder
- Topic — Construction of tree using traversal
- Topic — Binary Search tree
- Topic

Slide



Topic : Tree

Traversal
Construction of Binary tree from traversal
Binary Search Tree

Slide



Topic : Tree

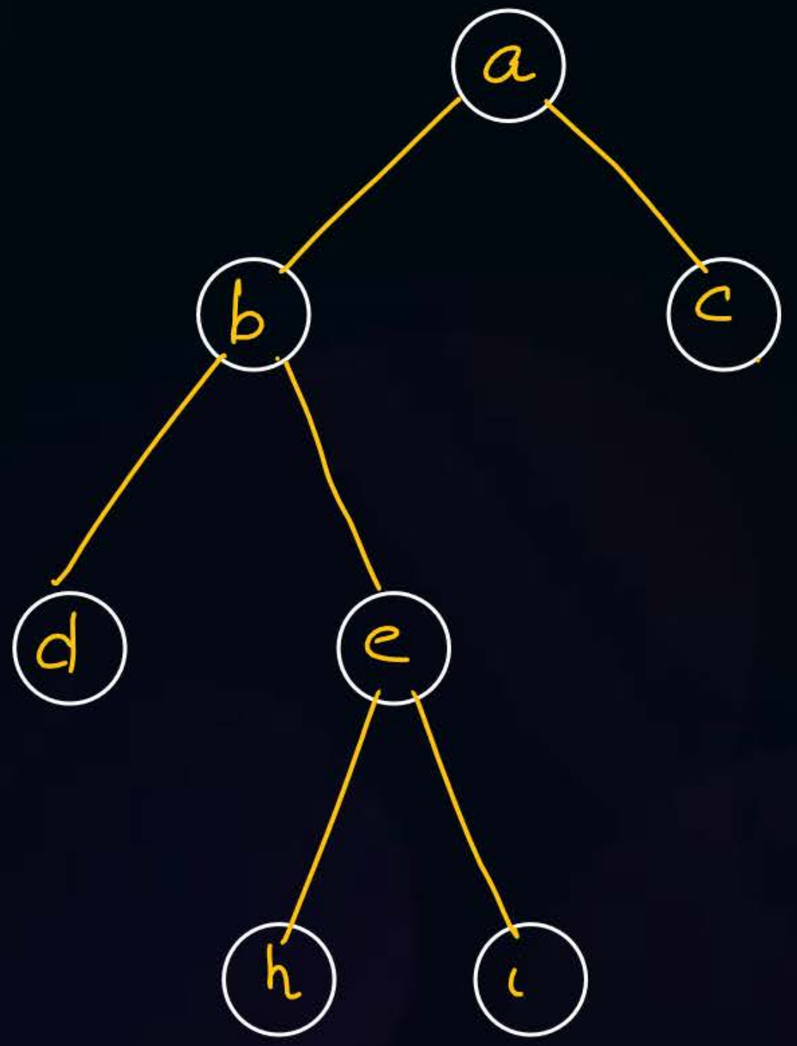
```
typedef  struct tnode{
        int data;
        struct tnode* left,
        struct tnode* right,
} tnode;
```



Slide



Topic : Tree



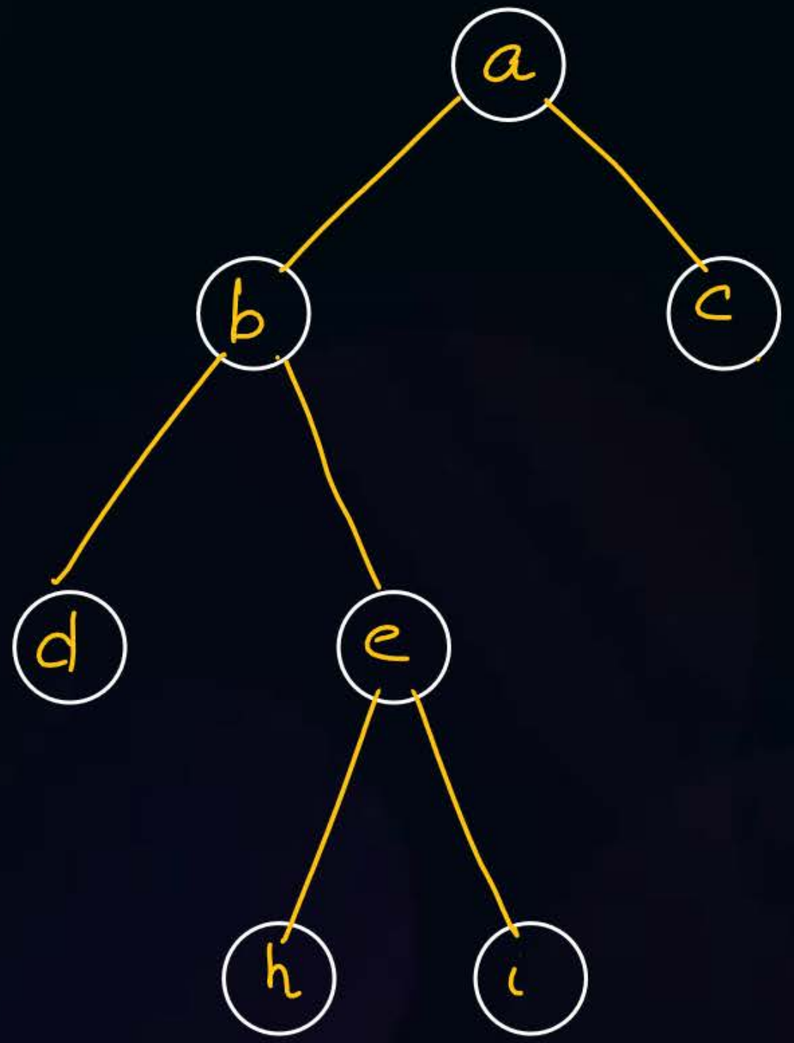
Traversal is process of visiting Node of binary tree.

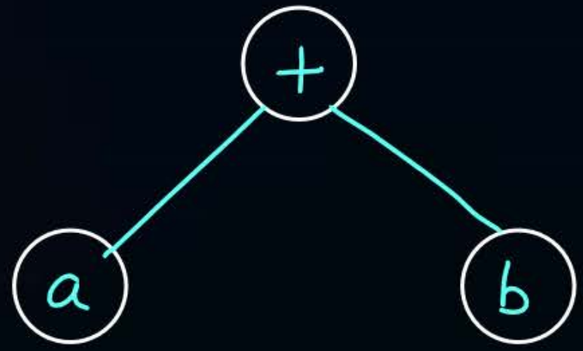
1. Inorder : Left Root Right
2. preorder : Root Left right
3. postorder : Left Right Root

Slide



Topic : Tree





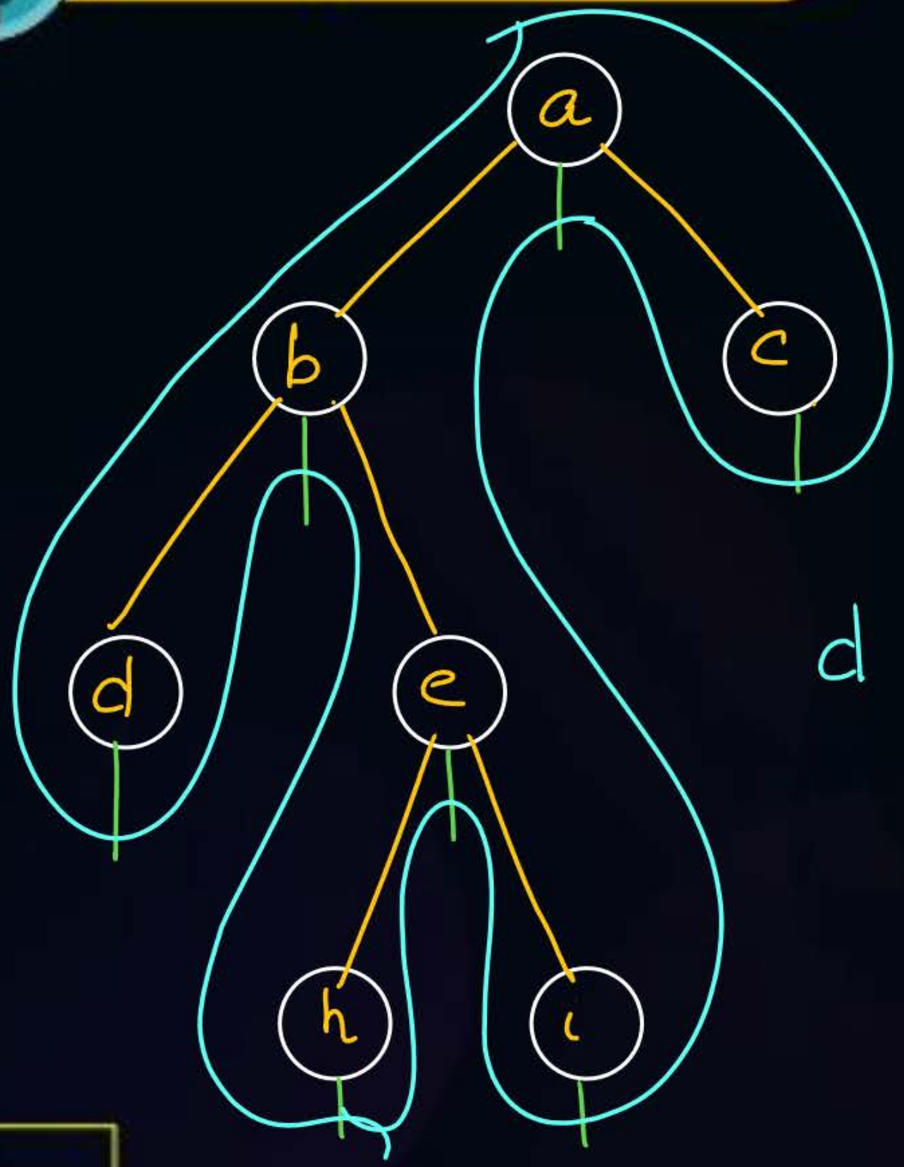
1 Inorder : a + b (Infix)

2. preorder : + a b (prefix)

3. postorder : a b + (postfix)

Slide

Topic : Tree





d b h e i a c

When tracing touches middle

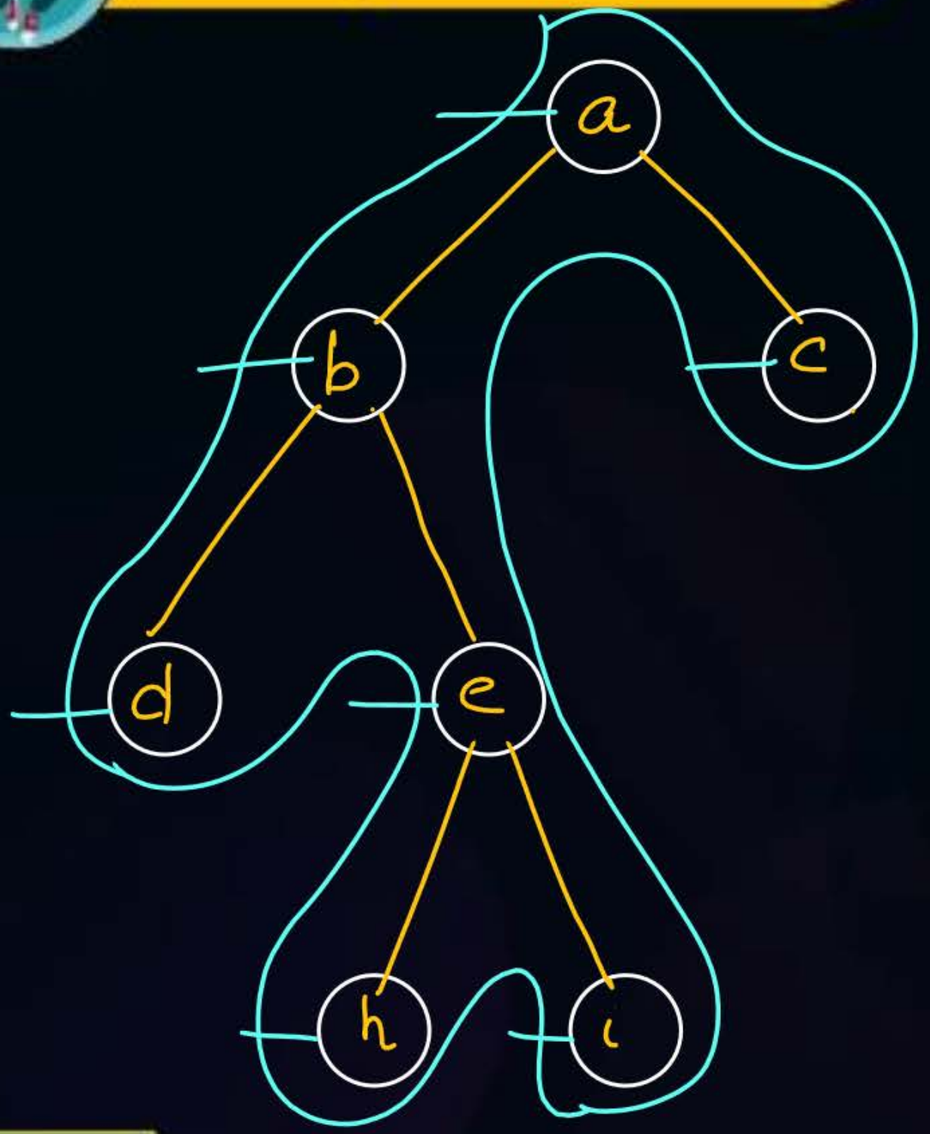
line and we perform print

then sequence will be inorder

Slide



Topic : Tree



preorder

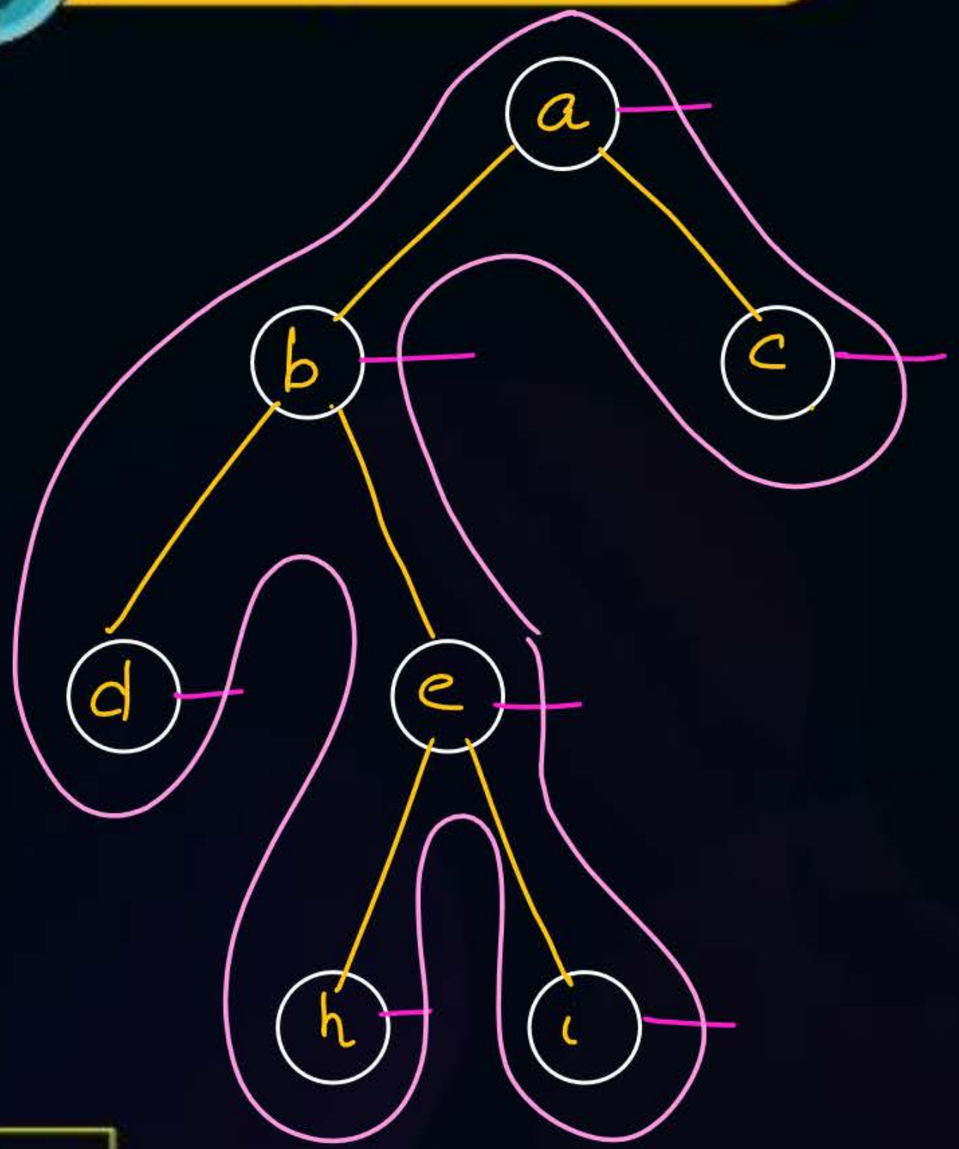
a b d e h i c

whenever the tracing touches left Line & we perform print operation then it generat preorder sequence

Slide



Topic : Tree



postorder traversal

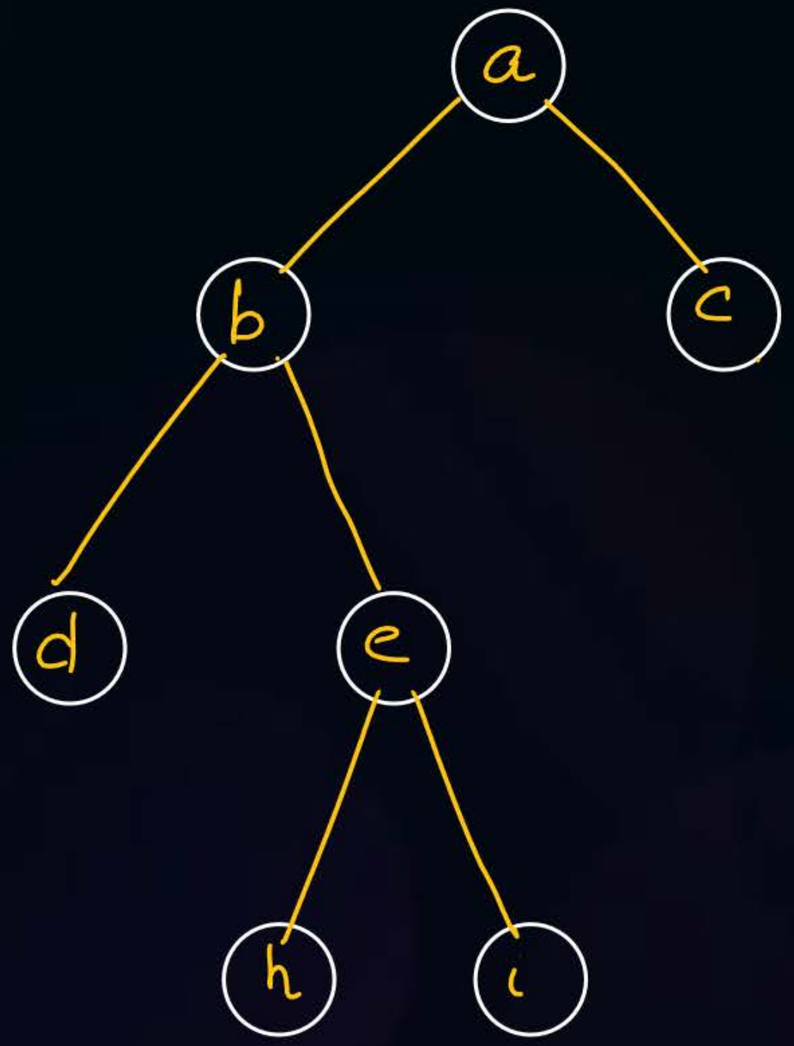
d h i e b c a

When ever tracing touches right line and we perform print operation then it generates post order sequence.

Slide



Topic : Tree



Traversal is process of visiting Node of binary tree.

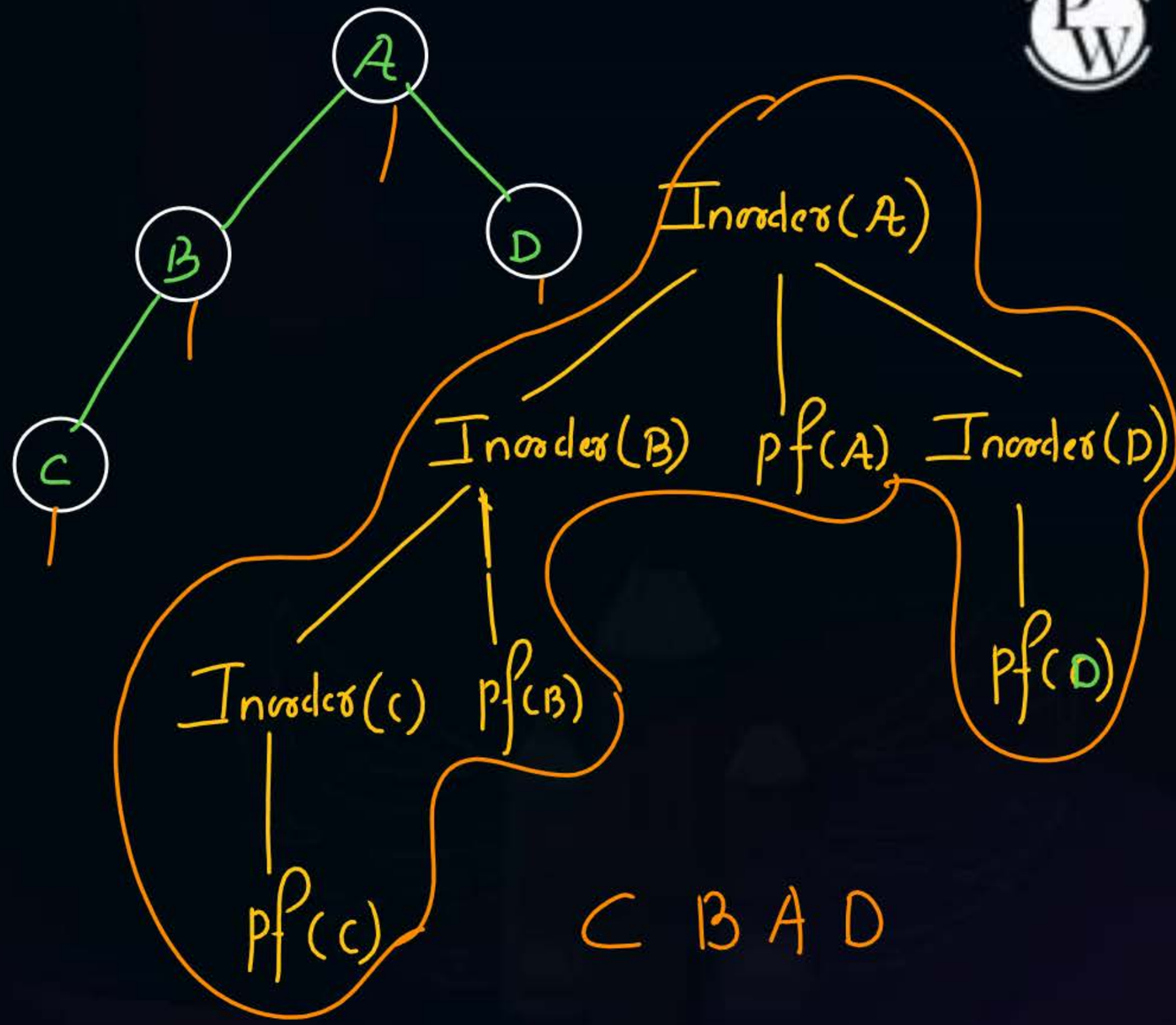
1. Inorder : Left Root Right
2. preorder : Root Left right
3. postorder : Left Right Root

Slide



Topic : Tree

```
void Inorder(tnode *t){
    if(t){
        Inorder(t->left)
        printf("%d", t->data);
        Inorder(t->right);
    }
}
```



Slide



Topic : Tree

```
void preorder(tnode *t) {
    if (t != NULL) {
        printf("%d", t->data);
        preorder(t->left);
        preorder(t->right);
    }
}
```

Slide



Topic : Tree

```
void postorder(tnode *t){
    if(t){
        postorder(t->left)
        postorder(t->right)
        printf("%d",t->data);
    }
}
```

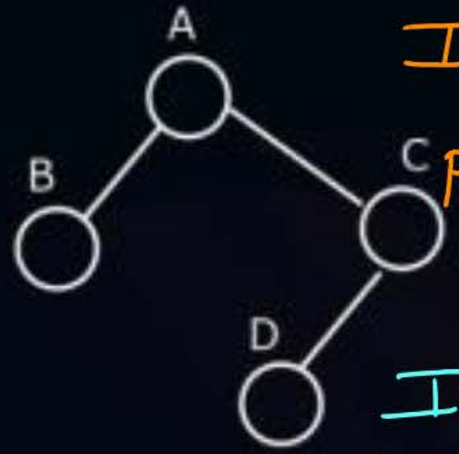
Slide



Topic : Question

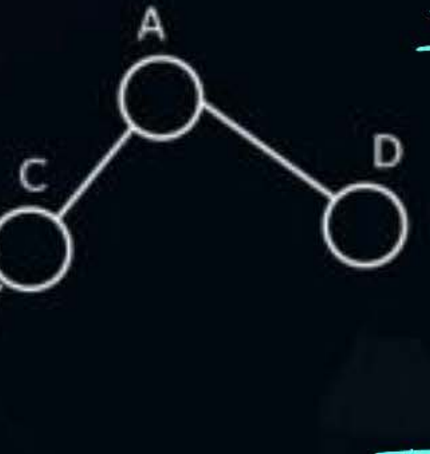
#Q. Which one of the following binary trees has its in-order and preorder traversals as BCAD and ABCD, respectively?

(A) ✗



Inorder : BADC
preorder : ABCD

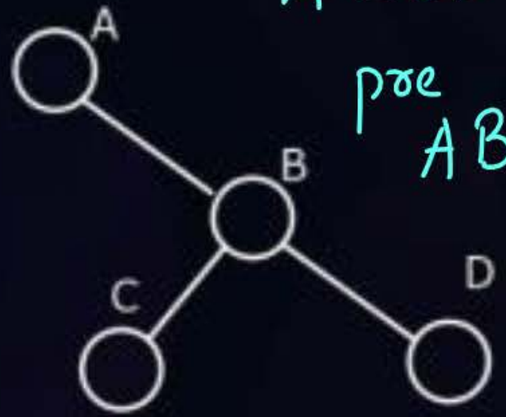
(B)



Inorder :
BCAD
preorder
ACBD

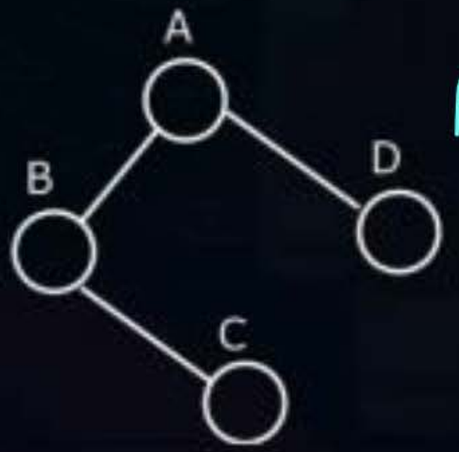
Inorder
ACBD

(C)



pre
ABCD

(D)



Inorder
BCAD
pre:
ABCD

Slide



Topic : Tree

Construction of tree using traversal

Inorder : a f b c d g e In

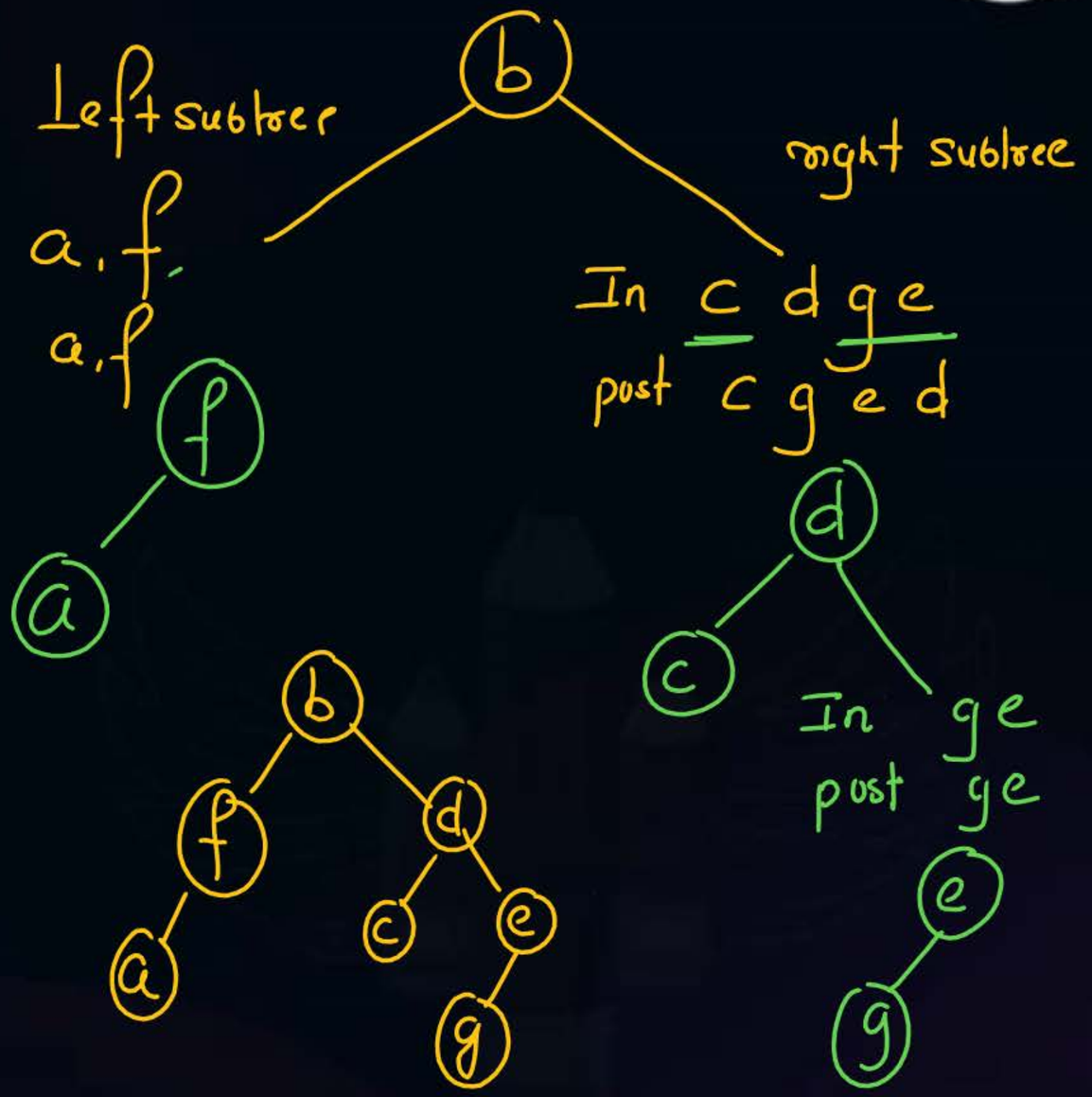
postorder : a f c g e d b ✓ post

Inorder : Left Root Right

postorder : Left Right Root

↑

Last element is Root



Slide



Topic : Tree

Inorder traversal

2, 7, 5, 6, 11, 1, 9, 5, 8

postorder

2, 5, 11, 6, 7, 5, 8, 9, 1

then sum of elements in leaf Node is ——

Slide



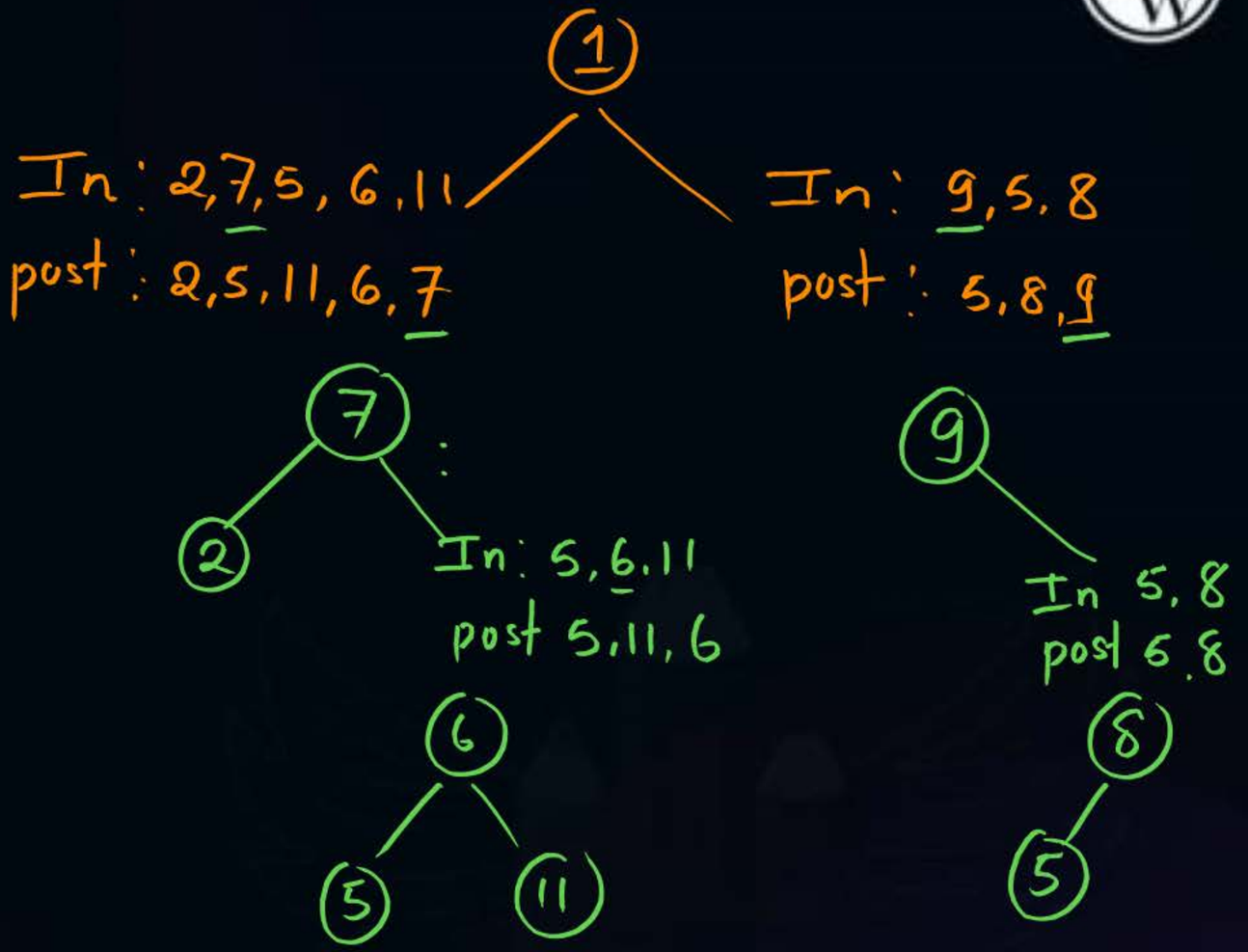
Topic : Tree

Inorder traversal

2, 7, 5, 6, 11, (1), 9, 5, 8

postorder

2, 5, 11, 6, 7 | 5, 8, 9 | 1



Slide



Topic : Tree

1. Identify the Root
2. goto Inorder traversal with Root identify No. of Nodes in Left subtree & right subtree
3. use step 2 Information to divide postorder in left and right subtree
4. generate the Subsequence In and postorder
5. Repeat the step 1 to 4 for left subtree & right subtree

Slide



Topic : Tree

Inorder

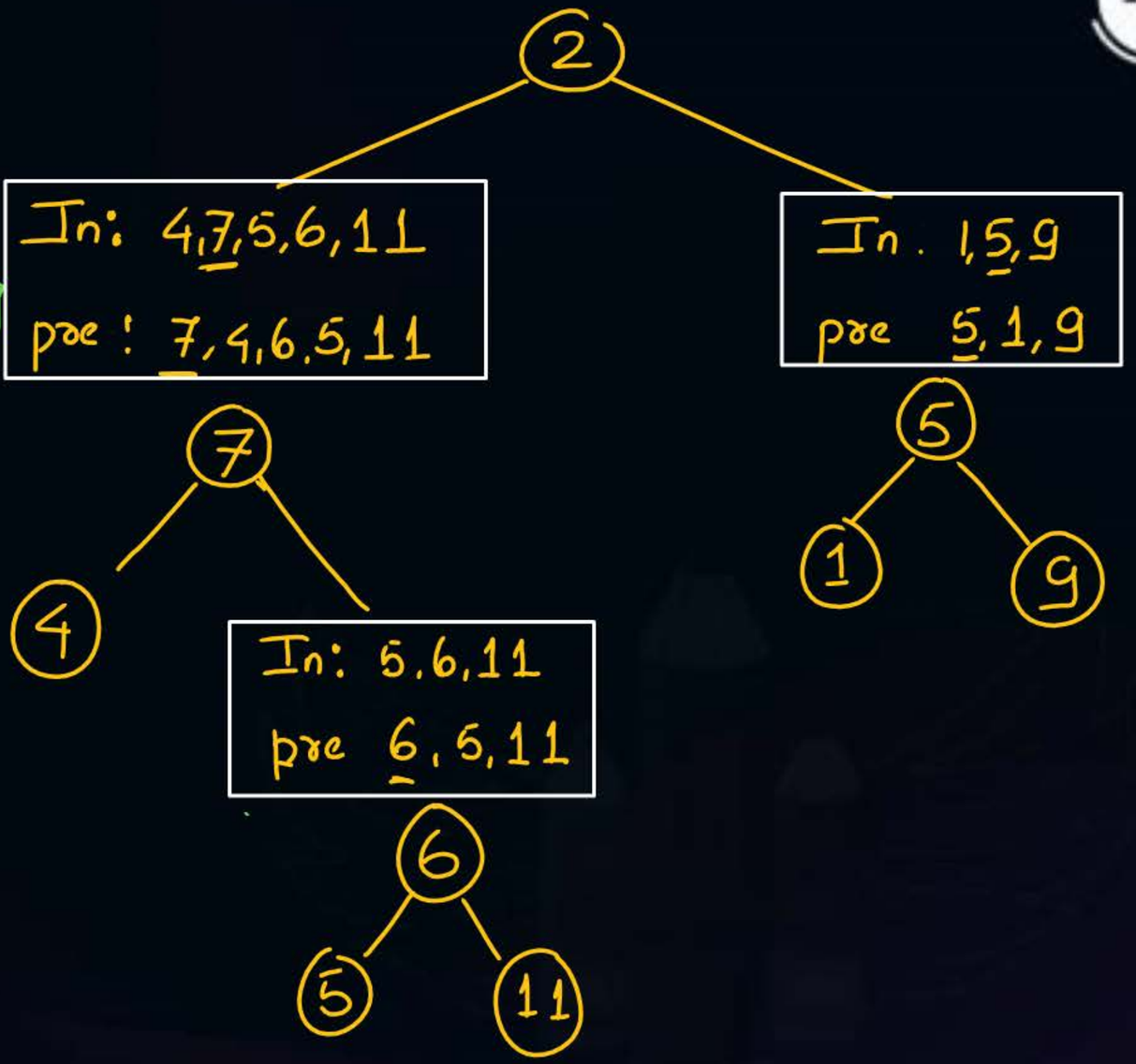
4, 7, 5, 6, 11, | 2 | 1, 5, 9

preorder :

2 | 7, 4, 6, 5, 11, | 5, 1, 9.

Inorder : Left Root Right

preorder : Root left right



Slide



Topic : Question

#Q.

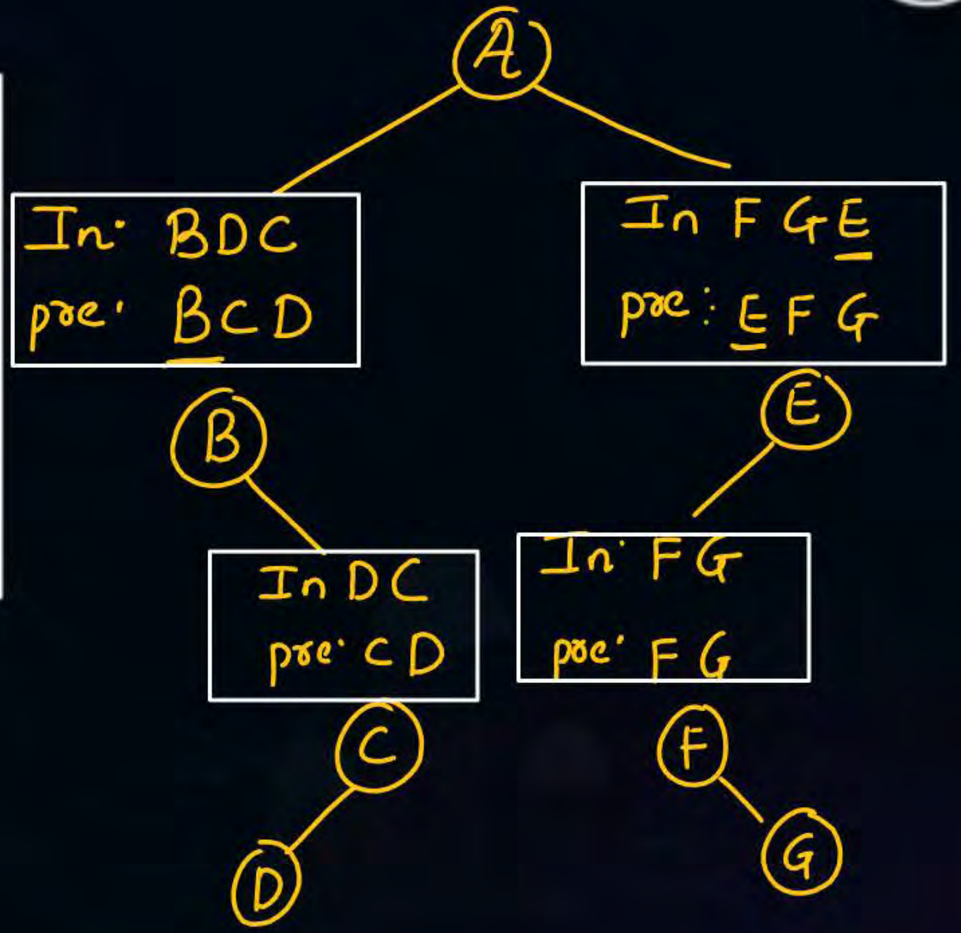
For given preorder : ABCDEFG

inorder : BDCAFGE

What is the postorder of a *Binary Tree*?

(a) CDBGFEA (b) DBCGFEA

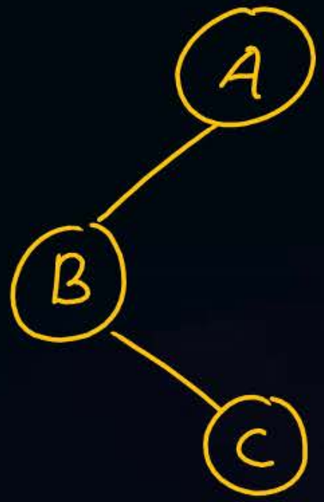
(c) DCBGFEA (d) None of these



Slide

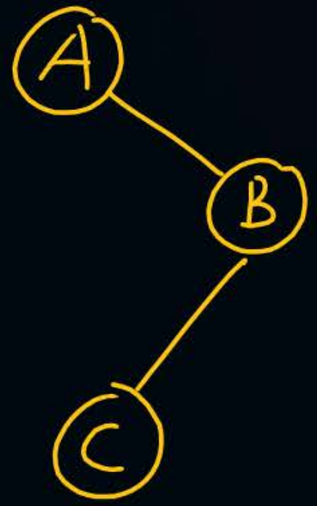


Topic : Tree



pre : A B C

post : C B A



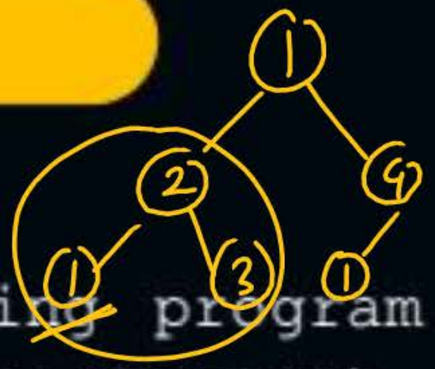
pre : A B C

post : C B A

for binary tree given post & pre order we can't uniquely construct the tree

Slide

Topic : Tree



$$1 + \underbrace{left()}_{1} + \underbrace{right}_{1}$$

$0 + left() + right()$ → $1 + 0 + 0$

$0 + NULL + NUL$

$0 + left() + right$ $+ 1 + NUL$

#Q.

Consider the following program the takes a tree root & and integer element as an argument

```
int count (struct node * node, int element) {
if (node == null)
    return 0;
else {
    int result = 0;
    if (node - > data == element)
        result = 1;
    return result + count(node-> left, element)
        + count (node - > right, element);
    }
}
```

(A) Count No. of leaves

(B) count No. of internal Node

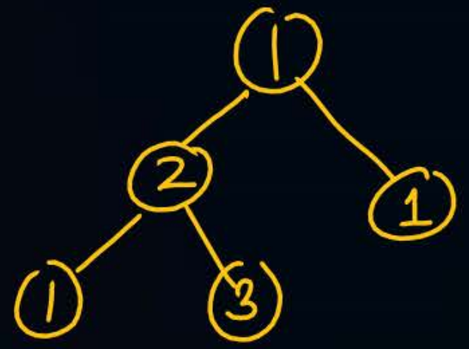
(C) Count No. of Nodes

(D) count frequency of element in tree

Slide



Topic : Tree



#Q.

Consider the following program the takes a tree root & and integer element as an argument

```
int count (struct node * node, int element) {
if (node == null)
    return 0;
else {
    int result = 0;
    if (node - > data == element)
            result = 1;
    return result + count(node-> left, element)
            + count (node - > right, element);
    }
}
```

count(t,1) = 3

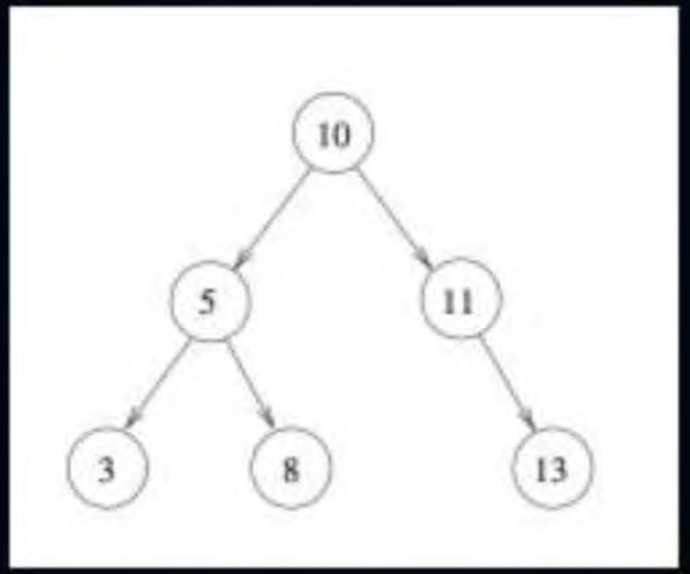
1 + count(2,1) + count(1,1)

0 + count(1,1) + count(3,1)



Topic : Question

Consider the function and the binary tree shown.



PYQ
2023

```
int foo(node *p) {
    int retval;
    if (p == NULL)
        return 0;
    else {
        retval = p->val + foo(p->left) + foo(p->right);
        printf("%d ", retval);
        return retval;
    }
}
```

A. 3 8 5 13 11 10
B. 3 5 8 10 11 13
C. 3 8 16 13 24 50
D. 3 16 8 50 24 13

Slide



2 mins Summary

Topic — Traversal

Topic — Construction of tree using pre & In

Topic — Construction of tree using post & In

Topic

Topic

Slide

THANK - YOU

Slide